UPDF
WWW.UPDF.COM

Diseño de Sistema RAG para Guía de Lanzarote

1. Modelo de Embedding

Elección: Hemos seleccionado intfloat/e5-base.

Justificación:

Tamaño y Rendimiento: Es un modelo relativamente ligero (base) pero extremadamente potente. Genera vectores de 768 dimensiones, lo que ofrece un equilibrio perfecto entre consumo de memoria RAM/CPU y precisión semántica. Funciona correctamente en español.

Naturaleza de la información: La guía de Lanzarote es puramente informativa (texto sobre lugares, horarios, descripciones). Un modelo de texto como este es una buena opción. Si en el futuro se quisiera que el usuario subiera una foto de la "Cueva de los Verdes" y preguntara "¿Qué es esto?", entonces sí necesitaríamos un modelo multimodal (como CLIP), pero para buscar en la guía turística, el texto es mucho más eficiente y preciso.

¿Fine-tuning? Inicialmente no. Los modelos e5 ya vienen bien entrenados para tareas asimétricas (pregunta corta -> documento largo).

¿Cuándo lo haría? Si notamos que el modelo falla con términos muy locales de Canarias (por ejemplo, confundir "guagua" con el llanto de un bebé en vez de un autobús, o no entender "Jameos").

¿Con qué datos? Crearía un dataset sintético de pares (Pregunta, Contexto Relevante) específicos de Lanzarote y aplicaría un fine-tuning usando técnicas como Contrastive Learning (Triplet Loss).

2. Diseño y Justificación de la Base de Datos Vectorial

Elección: Qdrant.

Justificación: Es un motor escrito en Rust, lo que lo hace rápido y eficiente en consumo de memoria. A diferencia de Pinecone (que es solo Cloud/Pago), Qdrant te permite levantar un contenedor Docker local y luego escalar a su versión Cloud si el proyecto crece. Soporta filtrado por metadatos (Payloads) de forma muy avanzada, lo cual es vital para RAG.

Tipo de Índice:

Inicialmente: Para un solo PDF de 40 páginas, Qdrant usará un indexado exacto (Brute-force) internamente porque el volumen es pequeño, garantizando un 100% de precisión.

A futuro: Si se escala la app para incluir todas las guías de Canarias y reseñas de hoteles, Qdrant activará automáticamente su índice HNSW (Hierarchical Navigable Small World, que es aproximado). HNSW sacrifica un ~1% de precisión a cambio de buscar en milisegundos entre millones de vectores.

Organización de los datos: En Qdrant, se organizan los vectores usando Payloads (Metadatos). No guardaría solo el chunk de texto, sino: {"content": "...", "fuente": "guia_lanzarote_2024.pdf", "municipio": "Haría", "categoria": "playa", "hash_documento": "1234abc"}. Esto permite hacer búsquedas híbridas (ej. "Busca por similitud semántica, pero SOLO donde categoria == playa").

3. Estrategia para Mantener la Información Actualizada

Para que la guía no se quede obsoleta:

Adquisición: En lugar de PDFs estáticos, conectaría un scraper a las webs de turismo locales o usaría Webhooks si se tiene un CMS (como WordPress). Cada vez que se publique un artículo nuevo, dispara un evento.

UPDF
WWW.UPDF.COM

Frecuencia: Basada en eventos (en tiempo real si cambian la web) o un Cron Job semanal que revise si hay PDFs nuevos en una carpeta de Google Drive / AWS S3.

Generación de embeddings: El texto nuevo pasa exactamente por el mismo pipeline: LangChain RecursiveCharacterTextSplitter -> HuggingFaceEmbeddings (e5-base).

Integración (Evitar duplicados): Esto es crucial. Al insertar en Qdrant (usando upsert), no dejar que Qdrant asigne IDs aleatorios. Genera un ID determinista (por ejemplo, un hash MD5 de la URL de origen + el número de chunk).

Si el texto de un lugar se actualiza, al generar el mismo ID y hacer upsert, Qdrant sobrescribe el vector antiguo en lugar de duplicarlo.

Si un documento se borra de la fuente original, se hace un client.delete() buscando por el metadato "fuente".

4. Integración con el Modelo de Lenguaje (LLM)

Elección del LLM: Gemini 2.5 Flash (a través de ChatGoogleGenerativeAI).

Justificación: Es recomendable ya que tiene una ventana de contexto masiva (por si recuperamos muchos chunks) y es extremadamente económico para tareas de RAG en producción en comparación con otros modelos de su misma categoría.

Flujo Completo:

1. El usuario envía: "¿Dónde puedo comer pescado?" a la API de FastAPI.
2. El modelo de embedding convierte esa frase en un vector de 768 números.
3. Qdrant busca en su espacio vectorial los 4 chunks más cercanos matemáticamente a ese vector.
4. FastAPI toma esos 4 textos recuperados y los inyecta en un Prompt Template junto con la pregunta original.
5. El LLM (Gemini) lee el contexto, formula una respuesta natural y FastAPI se la devuelve al usuario.

Optimización del Contexto:

Reranking: Después de que Qdrant devuelva 10 resultados, pasarlos por un modelo Cross-Encoder (como Cohere Rerank o BGE-Reranker) para reordenarlos y quedarte solo con los 3 más precisos antes de enviarlos al LLM. Esto reduce las alucinaciones enormemente.